
DSC 140B - The Final Project

Due: Wednesday, March 18

In GeoGuessr (<https://www.geoguessr.com>), players are shown a random image from Google Street View and guess where in the world it was taken. Pro players can often determine the location of an image with surprising accuracy using subtle visual cues like the style of road signs, the types of trees, or even the color of the sky (see, for example: https://youtube.com/shorts/hVM3GqNQE-c?si=Rc_byHTrbQ86nDK9).

In this project, you'll train a model to play SoCalGuessr, a GeoGuessr variant where all images are taken in Southern California. To make things a little easier, your model will only need to predict the city where an image was taken (Los Angeles, San Diego, SLO, Bakersfield, Riverside, or Anaheim) rather than the exact location.

The data.

The data for this project can be downloaded from:

<https://f000.backblazeb2.com/file/jeldridge-data/016-socal/data.zip>

This data makes up the *training set* for this project. Each image in the training set has a filename of the form: `<city>-<random-id>.jpg` where `<random-id>` is a random string of letters and digits, and `<city>` is the city where the image was taken. Since the goal is to train a model to predict the city, you can use the `<city>` part of the filename as the label for each image in the training set. The training set is (close to) balanced, with roughly the same number of images from each city.

The *test set* is withheld (you can't see it). When you submit your model, it will be evaluated on the test set, and your score will be the accuracy of your model on the test set (the percentage of images in the test set that your model correctly classifies). You may assume that the test set is also roughly balanced.

Your model.

Your model should be a neural network trained with gradient descent (or another iterative optimization procedure). You can use any architecture you like, and you can use any features you like. You can go *beyond* what we learned in class if you want. For example, we learned about convolutional neural networks (CNNs) this quarter, but you could use a more advanced architecture like a vision transformer. You may use pre-trained models if you'd like.

What to turn in.

You will turn in two things: 1) your model, and 2) a short report answering the specific questions below.

The model. You should turn in your model as a Python file named `predict.py` that contains a single function named `predict`. This function should take in a single argument, `image_path`, which is the path to a directory containing the images in the test set, and it should return a dictionary mapping each image filename in the test set to a predicted city.

Your `predict` function will be run on the Gradescope servers. You may assume that `numpy`, `pandas`, `scikit-learn`, `torch`, and `torchvision` are installed.

See the *example* section below for an example of what your `predict.py` file might look like. You do *not* need to turn in the code used to train your model.

You will turn in your model by uploading your `predict.py` file to the “Final Project - Model” assignment on Gradescope. The public autograder will check that your model's performance is above 40%, but it will not tell you what the test accuracy is exactly. This is to be more realistic: in the “real world”, you won't be able to see the test set, and even knowing that your model's true accuracy is above 40% isn't possible. To convince yourself that your model is performing well, you can evaluate it on a *validation set* as was done in DSC 80.

The report.

You should also turn in a short report (1-2 pages) with the following sections and content. Note: your report should have these exact section titles (bolded below) to make it easier for the TA to find them. You may type your report (it does *not* need to be handwritten).

1. **Human Baseline:** Report the accuracy a human achieves on this task. Do this by going to <https://eldridgejm.github.io/SoCalGuessr/> and playing the game yourself. Record your accuracy and a screenshot of the confusion matrix and include both in this section.
2. **Final Model Architecture:** Describe the architecture of your final model. At the least, this section must contain:
 - The number of hidden layers in your model, and how many units are in each hidden layer.
 - The number of parameters in your model.
 - The activations used in each layer.
3. **Training Procedure:** Describe the training process. At the least, this section should contain:
 - The training algorithm you used (E.g., SGD, Adam, etc.)
 - The loss function you used.
 - The time it took (in minutes, hours, etc.) for the training code to run.
 - A plot of the training curve, showing the iteration number versus the empirical risk.

Grading.

Your predictive model will be autograded and will receive a score based on its accuracy on the test set. In order to achieve full credit for this part of the project, your model must achieve at least a 70% accuracy on the test set. If your model achieve less than 70% accuracy, points will be deducted based on how far below 70% your model's accuracy is. The autograded part of the project is worth 50% of the total grade.

The other 50% of the grade will be based on the report you turn in. The report will be graded based on effort and completeness. To get full credit for the report, you should provide all of the required information in each section of the report.

The competition.

For fun, this project will also be a competition. The person who achieves the highest accuracy on the test set will win bragging rights and a small section of any letter of recommendation I write for them in the future describing this impressive achievement. The leaderboard will not be made public until after the project is due, however, I will post daily top accuracies on the messageboard starting 5 days before the project is due.

Example.

To help you get started, we have provided an example of a logistic regression model that achieves around 60% on the test set. This example shows two things that will likely be useful: 1) how to export a model's weights and then load them in the `predict` function, and 2) how to resize the images at train and test time. You may find the examples at:

- <http://dsc140b.com/materials/default/project/train.py>
- <http://dsc140b.com/materials/default/project/predict.py>

Note that you will only turn in the `predict.py` file, not the file that is used to train your model. Alongside your `.py` file, you will also likely turn in a file containing the weights of your model (e.g., a `.pt` file exported by your training code using `torch.save`). This file containing exported weights will be loaded by your `predict` function using `torch.load`. You can submit both `predict.py` and your model weights by zipping them and uploading the zip file to Gradescope.

Note: the combined size of your `predict.py` file and your model weights file must be less than 50MB. If your model is larger than 50MB, you will need to use a smaller model.